

Design Document

Personal Finance Manager

Console Application

Clean Code Learning Assignment

Prepared as part of structured software design evaluation

Document Version: 1.0
Date: May 2, 2026
Author: Vishwas Vijayvargiya
Project: Personal Finance Manager
Type: Console Application (.NET / C#)

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Design Philosophy	3
2	System Overview	3
2.1	High-Level Architecture	3
2.2	Technology Stack	4
3	Module and Class Design	4
3.1	Module Breakdown	4
3.2	Key Design Patterns	5
3.3	UML Class Diagram	5
3.4	Notable Relationships from the Diagram	6
4	Database Design	7
4.1	Overview	7
4.2	Database Schema Diagram	7
4.3	Table Definitions	7
4.3.1	Table: Categories	8
4.3.2	Table: Expenses	8
4.3.3	Table: Incomes	8
4.3.4	Table: Budgets	9
4.4	Cardinality Summary	9
4.5	Entity-Relationship Diagram	9
5	Application Flow	10
5.1	Overview	10
5.2	Startup Sequence	10
5.3	Expense Path (Option 1)	11
5.4	Income Path (Option 2)	11
5.5	Budget Path (Option 3)	11
5.6	Summary Path (Option 4)	11
6	Sequence Diagram	11
6.1	Add Expense – Full Interaction Sequence	11
6.2	Sequence Walk-through	12
7	Error Handling Strategy	13
7.1	Validation Layer	13
7.2	Database Failure Handling	13
7.3	Budget Over-limit Warning	14
8	Clean Code Adherence	14
9	Test Case Design	14
9.1	Expense Management	15

9.2	Income Management	15
9.3	Budget Management	16
9.4	Summary and Persistence	16
10	Setup Instructions	16
11	Evaluation Criteria Mapping	17
	Appendix: Diagram Index	17

1 Introduction

1.1 Purpose

This document describes the complete software design of the **Personal Finance Manager**, a console-based application developed as part of the Clean Code Learning Assignment. It covers the system architecture, class structure, database design, application flow, and interaction sequences – all grounded in the five diagrams produced for this project.

1.2 Scope

The application enables a user to manage personal financial records – expenses, income, and budgets – through a command-line interface backed by a relational database using **Entity Framework Core**. All data persists across application restarts via **AppDbContext** and SQL Server.

1.3 Design Philosophy

The system is built around the following principles:

- **Separation of Concerns** – distinct layers for UI (Controllers and ConsoleHelper), business logic (Services), and data access (Repositories).
- **Dependency Inversion** – controllers depend on service interfaces such as **IExpenseService** and **IBudgetService**, never on concrete classes.
- **Factory Pattern** – **ServiceFactory** resolves service instances from the DI container at runtime, keeping controllers loosely coupled.
- **Repository Pattern** – a generic **Repository<T>** base provides CRUD; specialised repositories extend it with domain-specific queries.
- **Don't Repeat Yourself (DRY)** – shared utilities (**ConsoleHelper** and **InputValidator**) eliminate duplication across all controllers.
- **Observer-like Budget Tracking** – after every expense is saved, **BudgetService.UpdateSpentAmountAsync** is automatically called to keep the budget's **SpentAmount** in sync.

2 System Overview

2.1 High-Level Architecture

The application follows a **three-tier layered architecture** in C# / .NET:

1. **Presentation Layer** – **MenuController**, **ExpenseController**, **IncomeController**, **BudgetController**, **SummaryController**, and **ConsoleHelper**.
2. **Business Logic Layer** – **ExpenseService**, **IncomeService**, **BudgetService**, and **SummaryService**, each behind a typed interface.

3. **Data Access Layer** – `ExpenseRepository`, `IncomeRepository`, and `BudgetRepository` extending the generic `Repository<T>`; plus `AppDbContext` with three EF Core configuration classes.

Application startup (`Program.cs`) registers all services and the `AppDbContext` with the DI container, then calls `MenuController.ShowMenu()` to begin the main interaction loop.

2.2 Technology Stack

Component	Choice
Language	C# (.NET 7 / .NET 8)
ORM / DB Access	Entity Framework Core
Database	SQL Server (relational, persistent)
Interface	Console (stdin / stdout)
DI Container	Microsoft.Extensions.DependencyInjection
Test Framework	xUnit / NUnit

3 Module and Class Design

3.1 Module Breakdown

Module / Namespace	Responsibility
Controllers	<code>MenuController</code> , <code>ExpenseController</code> , <code>IncomeController</code> , <code>BudgetController</code> , <code>SummaryController</code> – route user input to the appropriate service
Services	Business logic interfaces and implementations: <code>IExpenseService</code> / <code>ExpenseService</code> , <code>IIncomeService</code> / <code>IncomeService</code> , <code>IBudgetService</code> / <code>BudgetService</code> , <code>ISummaryService</code> / <code>SummaryService</code>
Repositories	Generic <code>Repository<T></code> (abstract) and specialised <code>ExpenseRepository</code> , <code>IncomeRepository</code> , <code>BudgetRepository</code>
Models	Entity classes (<code>Expense</code> , <code>Income</code> , <code>Budget</code> , <code>Category</code>) and record DTOs (<code>ExpenseDto</code> , <code>IncomeDto</code> , <code>BudgetDto</code>)
Data	<code>AppDbContext</code> , <code>ExpenseConfiguration</code> , <code>IncomeConfiguration</code> , <code>BudgetConfiguration</code>
Helpers	<code>ConsoleHelper</code> – <code>ShowMenu</code> , <code>ShowTable</code> , <code>ShowError</code> , <code>ShowSuccess</code>
Validators	<code>InputValidator</code> – <code>ValidateExpense</code> , <code>ValidateIncome</code> , <code>ValidateBudgetDto</code>
Factory	<code>ServiceFactory</code> – <code>Create(IServiceProvider, T)</code>

3.2 Key Design Patterns

- **Factory Pattern:** `ServiceFactory.Create(provider)` resolves `IExpenseService` from the DI container at runtime, as annotated on the class diagram: *“Factory pattern resolves IExpenseService from DI container”*.
- **Repository Pattern:** `ExpenseService` holds only `IExpenseRepository` and never touches `AppDbContext` directly, as annotated on the sequence diagram: *“Service only knows IExpenseRepository”*.
- **Observer-like Update:** After a successful expense save, `ExpenseService` triggers `BudgetService.UpdateSpentAmountAsync(categoryId)` – an event-like cascade without a formal event system.
- **Generic Repository:** The abstract `Repository<T>` owns the `AppDbContext` and a typed `DbSet<T>`, providing `AddAsync`, `GetByIdAsync`, `GetAllAsync`, `UpdateAsync`, and `DeleteAsync` for every entity type.

3.3 UML Class Diagram

The diagram below shows the full class hierarchy: entity models, record DTOs, service interfaces and implementations, repository interfaces and concrete classes, `AppDbContext`, EF configuration classes, and utility / helper classes.

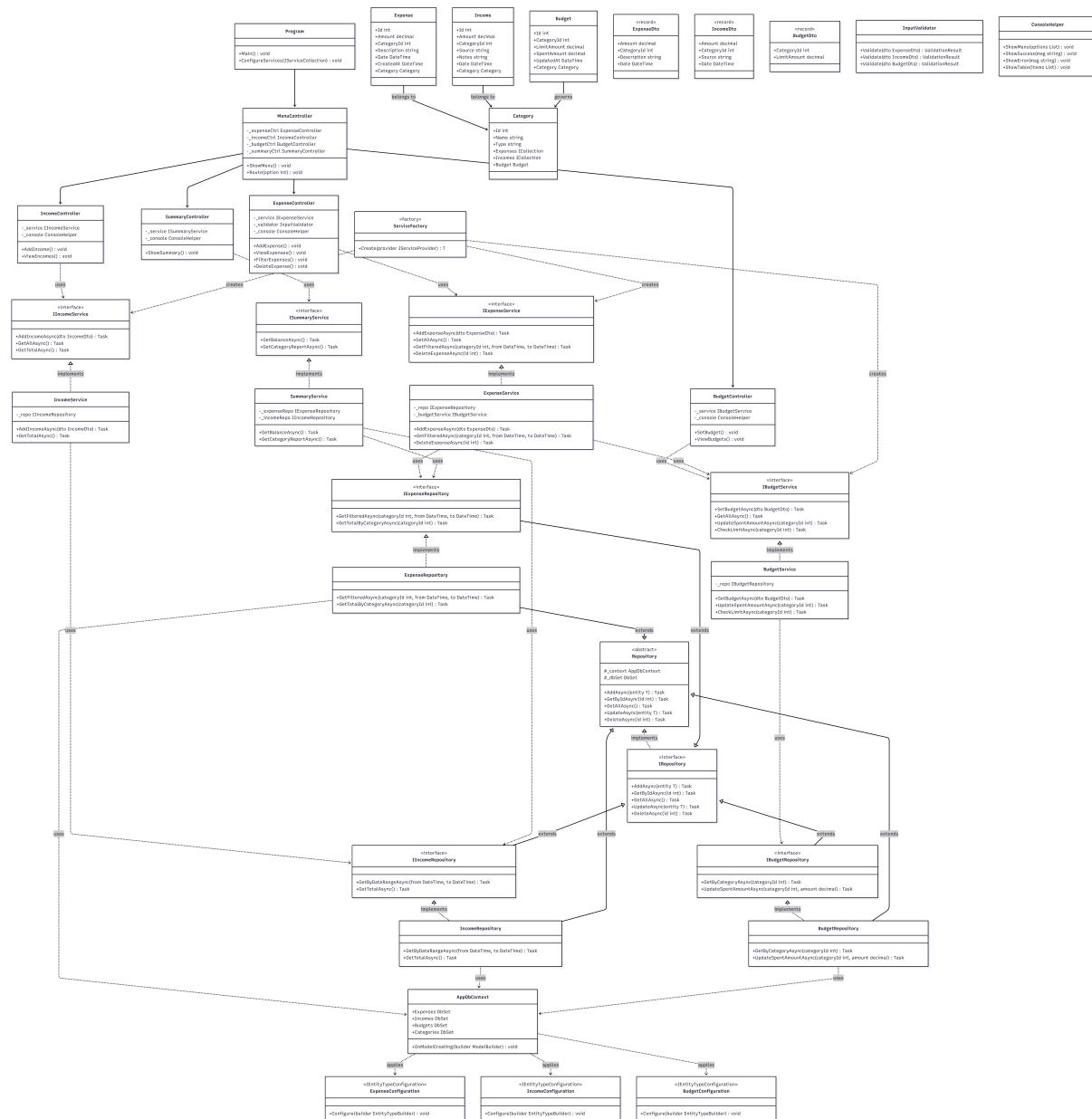


Figure 1: UML Class Diagram – Full System

3.4 Notable Relationships from the Diagram

- MenuController holds references to all four child controllers and exposes ShowMenu() and RouteOption(int).
- ExpenseController depends on IExpenseService and InputValidator; exposes AddExpense(), ViewExpenses(), FilterExpenses(), and DeleteExpenses().
- SummaryController holds an ISummaryService reference and exposes ShowSummary().
- BudgetController depends on IBudgetService and exposes SetBudget() and ViewBudgets().

- `IExpenseRepository` additionally exposes `GetFilteredAsync(categoryId, from, to)` and `GetTotalByCategoryAsync(categoryId)`.
- `IIncomeRepository` extends `IRepository` with `GetByDateRangeAsync(from, to)` and `GetTotalAsync()`.
- `IBudgetRepository` extends `IRepository` with `GetByCategoryAsync(categoryId)` and `UpdateSpentAmountAsync(categoryId, amount)`.
- `Category` is the central shared entity; `Expense`, `Income`, and `Budget` all carry a `CategoryId` FK.
- `AppDbContext` owns `DbSet` properties for all four entities and overrides `OnModelCreating(ModelBuilder)`.

4 Database Design

4.1 Overview

The database contains four tables: `Categories`, `Expenses`, `Incomes`, and `Budgets`. `Categories` is the central lookup table; the other three hold a `CategoryId` foreign key. EF Core `IEntityTypeConfiguration<T>` classes manage all column mappings and constraints via the Fluent API, keeping model classes annotation-free.

4.2 Database Schema Diagram

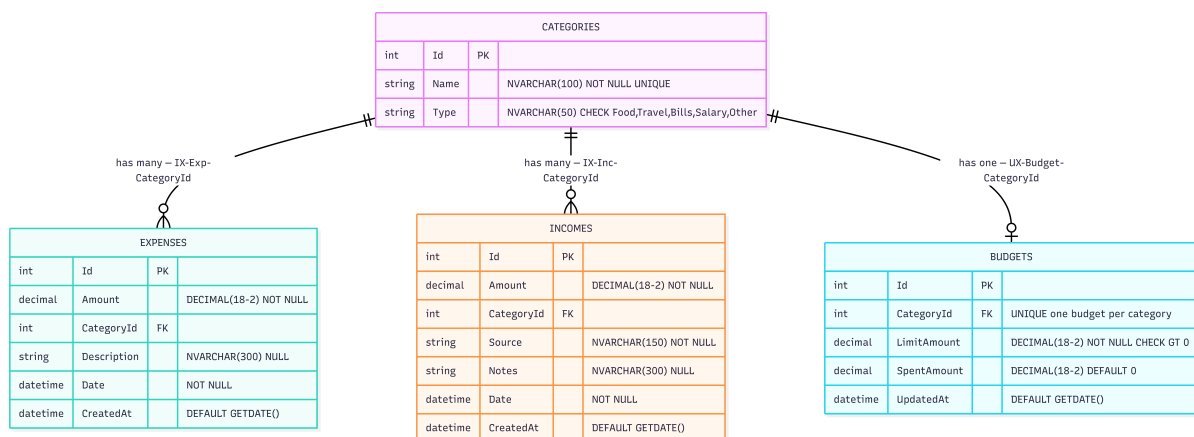


Figure 2: Relational Database Schema Diagram

4.3 Table Definitions

4.3.1 Table: Categories

Column	Type	Constraint	Notes
Id	int	PK, AUTOINCREMENT	
Name	NVARCHAR(100)	NOT NULL, UNIQUE	
Type	NVARCHAR(50)	CHECK: Food, Travel, Bills, Salary, Other	

4.3.2 Table: Expenses

Column	Type	Constraint	Notes
Id	int	PK, AUTOINCREMENT	
Amount	DECIMAL(18,2)	NOT NULL	
CategoryId	int	FK to Categories	IX-Exp-CategoryId
Description	NVARCHAR(300)	NULL	
Date	datetime	NOT NULL	
CreatedAt	datetime	DEFAULT GET-DATE()	

4.3.3 Table: Incomes

Column	Type	Constraint	Notes
Id	int	PK, AUTOINCREMENT	
Amount	DECIMAL(18,2)	NOT NULL	
CategoryId	int	FK to Categories	IX-Inc-CategoryId
Source	NVARCHAR(150)	NOT NULL	
Notes	NVARCHAR(300)	NULL	
Date	datetime	NOT NULL	
CreatedAt	datetime	DEFAULT GET-DATE()	

4.3.4 Table: Budgets

Column	Type	Constraint	Notes
Id	int	PK, AUTOINCREMENT	
CategoryId	int	FK to Categories, UNIQUE	UX-Budget-CategoryId
LimitAmount	DECIMAL(18,2)	NOT NULL, CHECK > 0	
SpentAmount	DECIMAL(18,2)	DEFAULT 0	Derived; updated on expense add
UpdatedAt	datetime	DEFAULT GET-DATE()	

4.4 Cardinality Summary

- **Category to Expenses:** one-to-many.
- **Category to Incomes:** one-to-many.
- **Category to Budget:** one-to-one (unique budget per category).

4.5 Entity-Relationship Diagram

The ER diagram uses Chen notation: rectangles for entities, diamonds for relationships, ovals for attributes, underlined ovals for primary keys, and a dashed oval for the derived attribute `Budget.SpentAmount`, which is recomputed each time an expense is added.

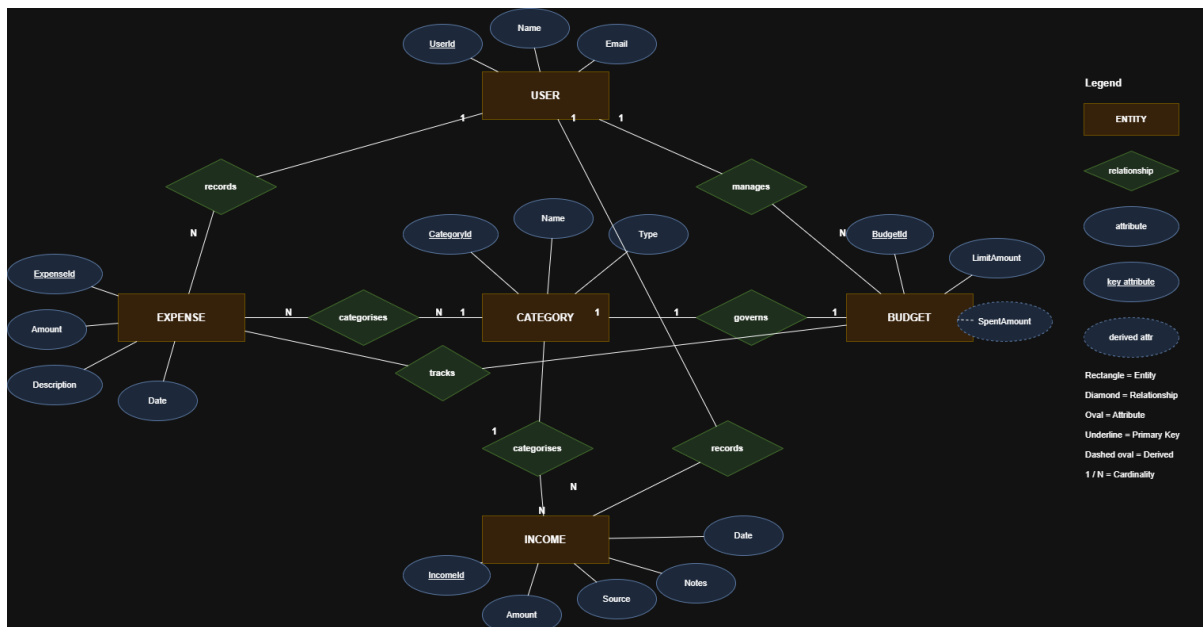


Figure 3: Entity-Relationship (ER) Diagram – Chen Notation

Key relationships visible in the diagram:

- **USER** *records* **EXPENSE** (1:N) and *manages* **BUDGET** (1:N).
- **CATEGORY** *categorises* **EXPENSE** (1:N) and *categorises* **INCOME** (1:N).
- **CATEGORY** *governs* **BUDGET** (1:1).
- **CATEGORY** *tracks* **INCOME** (1:N).

5 Application Flow

5.1 Overview

The flow diagram captures the complete runtime behaviour from `Program.cs` startup through every menu-selectable path, including input validation branches, service calls, repository persistence, and the post-save budget update step.

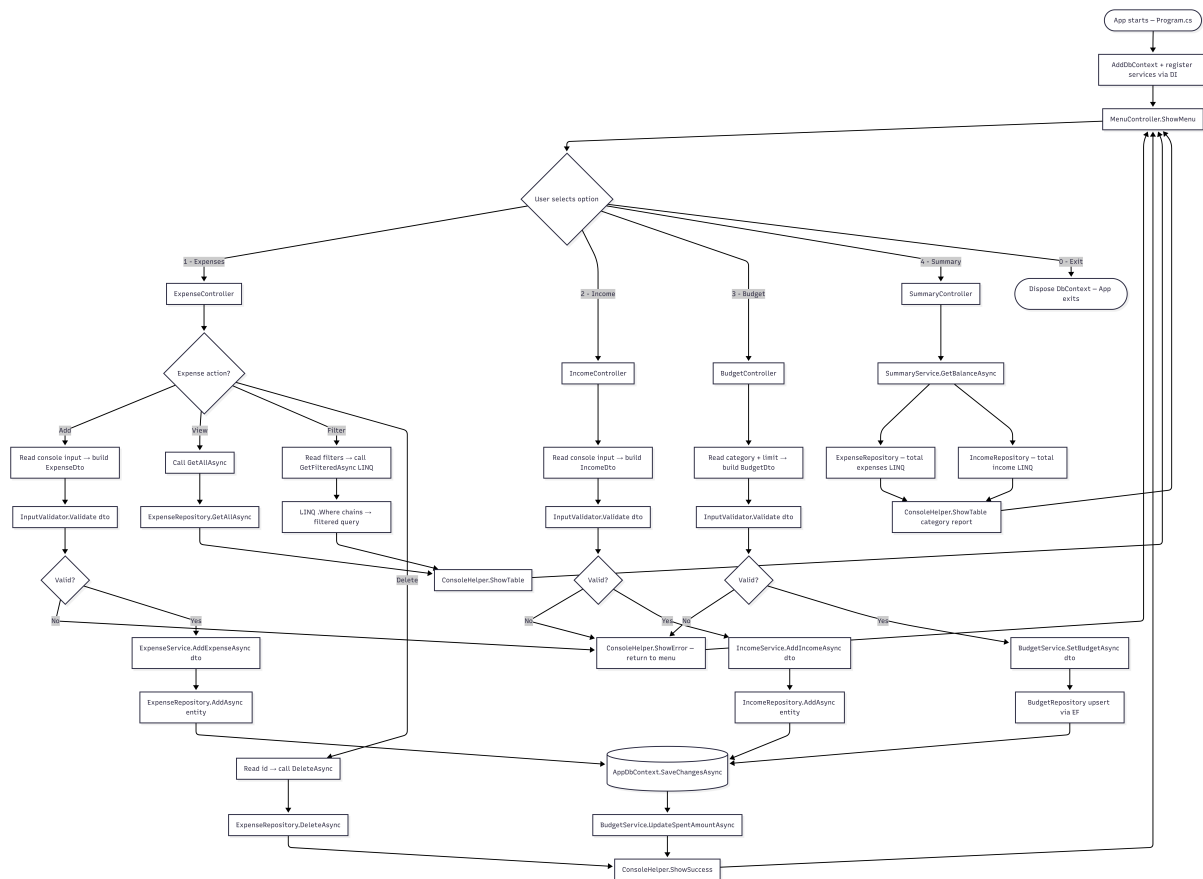


Figure 4: Full Application Flow Diagram

5.2 Startup Sequence

1. `Program.cs` runs: `AddDbContext` and registers all services via DI.
2. `MenuController.ShowMenu()` enters the main interaction loop.
3. The user selects one of five options: **1 - Expenses**, **2 - Income**, **3 - Budget**, **4 - Summary**, or **0 - Exit**.

4. On exit (option 0), the `DbContext` is disposed and the app exits cleanly.

5.3 Expense Path (Option 1)

1. `ExpenseController` is invoked; the user selects **Add**, **View**, **Filter**, or **Delete**.
2. **Add**: Console input is collected and built into an `ExpenseDto`. `InputValidator` checks it – on failure, an error is shown and the menu returns; on success, `ExpenseService.AddExpenseAsync` is called, which calls `ExpenseRepository.AddAsync`, then `SaveChangesAsync`, then `BudgetService.UpdateSpentAmountAsync`, and finally shows success.
3. **View**: `ExpenseRepository.GetAllAsync` is called and results are rendered in a table via `ConsoleHelper.ShowTable`.
4. **Filter**: Category and date-range inputs drive a LINQ `.Where` chain in `GetFilteredAsync`, with results shown in a table.
5. **Delete**: The user provides an ID; `ExpenseRepository.DeleteAsync` removes the record and `SaveChangesAsync` persists the change.

5.4 Income Path (Option 2)

Console input is collected and built into an `IncomeDto`. After successful validation, `IncomeService.AddIncomeAsync` is called, which delegates to `IncomeRepository.AddAsync` and `SaveChangesAsync`. Success is confirmed via `ConsoleHelper.ShowSuccess`.

5.5 Budget Path (Option 3)

The user provides a category and a limit amount, built into a `BudgetDto`. After validation, `BudgetService.SetBudgetAsync` performs an upsert via EF Core (insert if new, update if existing) and persists with `SaveChangesAsync`.

5.6 Summary Path (Option 4)

`SummaryController` calls `SummaryService.GetBalanceAsync`, which runs two parallel LINQ aggregations – total expenses and total income – against their respective repositories. The results are formatted and displayed as a category-wise spending and balance report via `ConsoleHelper.ShowTable`.

6 Sequence Diagram

6.1 Add Expense – Full Interaction Sequence

The sequence diagram below shows the complete end-to-end interaction for the **Add Expense** operation, covering validation, factory resolution, repository persistence, EF Core save, and the observer-like budget update.

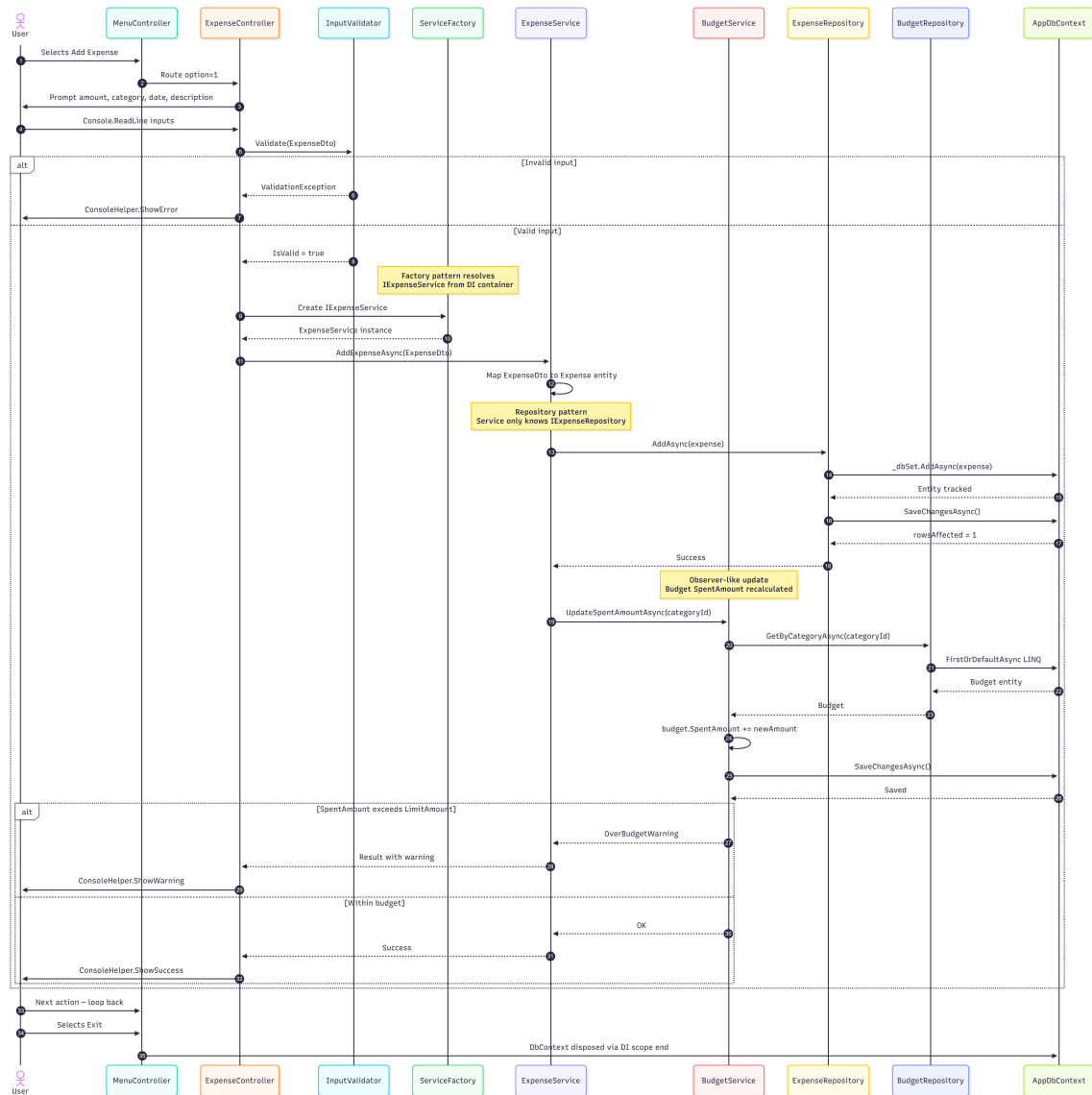


Figure 5: Sequence Diagram – Add Expense (including Budget SpentAmount update)

6.2 Sequence Walk-through

1. The **User** selects “Add Expense”. `MenuController` calls `RouteOption(1)`, which delegates to `ExpenseController`.
2. `ExpenseController` prompts for amount, category, date, and description via `Console.ReadLine`.
3. `InputValidator.Validate(ExpenseDto)` is called.
 - *Invalid input*: a `ValidationException` is returned; `ConsoleHelper.ShowError` displays the message and flow returns to menu.
 - *Valid input*: `IsValid = true` and flow continues.
4. `ServiceFactory` resolves `IExpenseService` from the DI container (*Factory Pattern*) and returns the `ExpenseService` instance.

5. `ExpenseController` calls `AddExpenseAsync(ExpenseDto)`.
6. `ExpenseService` maps the DTO to an `Expense` entity and calls `IExpenseRepository.AddAsync(expense)` (*Repository Pattern* – the service only knows the interface, never the concrete class).
7. `ExpenseRepository` calls `_dbSet.AddAsync` (entity is tracked), then `SaveChangesAsync()` returning `rowsAffected = 1`.
8. **Observer-like budget update:** `ExpenseService` calls `BudgetService.UpdateSpentAmountAsync`
 - `BudgetRepository.GetByCategoryAsync` runs `FirstOrDefaultAsync` and returns the `Budget` entity.
 - `budget.SpentAmount += newAmount` is computed in memory.
 - `SaveChangesAsync()` persists the updated budget.
9. *SpentAmount exceeds LimitAmount:* an `OverBudgetWarning` is propagated back and `ConsoleHelper.ShowWarning` is displayed. The expense is still saved regardless.
10. *Within budget:* `ConsoleHelper.ShowSuccess` is displayed.
11. The loop returns to `MenuController.ShowMenu()` for the next action.
12. On “Select Exit”, the `DbContext` is disposed via DI scope end.

7 Error Handling Strategy

7.1 Validation Layer

`InputValidator` intercepts all user input before it reaches the service layer:

- **ValidateExpense:** amount must be greater than zero; category ID must exist; date must be valid; description must be at most 300 characters.
- **ValidateIncome:** amount greater than zero; source non-empty and at most 150 characters; valid category ID and date.
- **ValidateBudgetDto:** limit amount greater than zero; valid category ID.
- All methods return a typed `ValidationResult`; errors are displayed via `ConsoleHelper.ShowError` without exposing stack traces.

7.2 Database Failure Handling

- The DI-scoped `AppDbContext` is disposed cleanly on exit or on an unhandled exception.
- All repository `async` calls are awaited inside `try-catch` blocks in the service layer; exceptions surface as friendly console messages.
- EF Core transactions wrap write operations to prevent partial commits.
- Connection failures at startup are caught in `Program.cs` and reported before exit.

7.3 Budget Over-limit Warning

After `UpdateSpentAmountAsync` completes, `BudgetService.CheckLimitAsync` compares `SpentAmount` against `LimitAmount`. If the limit is exceeded, an `OverBudgetWarning` is returned and displayed to the user. The expense is persisted regardless, giving the user full visibility without blocking their workflow.

8 Clean Code Adherence

Principle	Implementation
Meaningful Names	All classes and methods use full, self-documenting identifiers such as <code>AddExpenseAsync</code> , <code>GetFilteredAsync</code> , and <code>UpdateSpentAmountAsync</code> . No abbreviations are used.
Small Functions	Controller methods only collect input and delegate; service methods only orchestrate; repository methods only query. No method has more than one responsibility.
No Duplication	<code>ConsoleHelper</code> centralises all display logic; <code>InputValidator</code> centralises all validation; the generic <code>Repository<T></code> eliminates per-entity CRUD boilerplate.
Dependency Inversion	All cross-layer dependencies target interfaces injected via the DI container. No controller or service instantiates a concrete dependency directly.
Factory Pattern	<code>ServiceFactory.Create(IServiceProvider, T)</code> resolves services at runtime, removing direct instantiation from controllers.
No Hardcoding	Category types are database-driven; connection strings live in <code>appsettings.json</code> ; all magic strings and numbers are named constants.
Error Handling	<code>ValidationResult</code> objects carry structured messages; raw stack traces never reach the console.
EF Configuration	Fluent API in <code>IEntityTypeConfiguration<T></code> classes keeps model classes annotation-free and easy to read.

9 Test Case Design

9.1 Expense Management

#	Scenario	Input	Expected Output
1	Add valid expense	Amount=500, Category=Food, Date=today	Record saved; Show-Success
2	Add with amount = 0	Amount=0	ValidationResult error; ShowError
3	Add with negative amount	Amount=-100	ValidationResult error; ShowError
4	Add with invalid date	Date=32/13/2024	ValidationResult error; ShowError
5	Add with empty description	Description empty	Record saved (nullable field)
6	Delete existing expense	ID=1	Record removed; ShowSuccess
7	Delete non-existent expense	ID=9999	Not found error; ShowError
8	Filter by valid category	CategoryId=1	Matching expenses listed
9	Filter by date range	From/To within January 2024	Only January records returned
10	View all expenses on empty DB	No records	Empty table shown; no crash

9.2 Income Management

#	Scenario	Input	Expected Output
11	Add valid income	Amount=10000, Source=Salary	Record saved; Show-Success
12	Add with empty source	Source empty	ValidationResult error; ShowError
13	Add with source exceeding 150 chars	Overly long string	ValidationResult error
14	View all income entries	Multiple records	All records displayed in table

9.3 Budget Management

#	Scenario	Input	Expected Output
15	Set budget for a category	Category=Food, Limit=3000	Budget saved (upsert)
16	Update existing budget limit	Category=Food, Limit=5000	LimitAmount updated
17	Set budget with limit = 0	Limit=0	ValidationResult error
18	Expense pushes SpentAmount over LimitAmount	Expense=3500 vs Limit=3000	ShowWarning displayed
19	View all budgets	Multiple categories	Table with Limit and SpentAmount

9.4 Summary and Persistence

#	Scenario	Input	Expected Output
20	Summary with income and expenses	Multiple records	Correct balance; per-category totals
21	Summary with zero records	Empty database	Balance = 0; no crash
22	Data persists after restart	Add record; restart app	Record still present
23	DB connection unavailable	Simulate failure	User-friendly error; app does not crash
24	GetTotalAsync on empty income table	No income records	Returns 0; no exception

10 Setup Instructions

1. **Prerequisites:** Install .NET 7 SDK (or .NET 8) and SQL Server or SQLite.
2. **Clone the repository:**

```
1 git clone https://github.com/<your-repo>/personal-finance-
  manager.git
2 cd personal-finance-manager
```

3. **Configure the connection string in appsettings.json:**

```
1 {
2   "ConnectionStrings": {
3     "DefaultConnection":
4       "Server=localhost;Database=FinanceDb;Trusted_Connection=
        True;"
5   }
6 }
```

4. Apply EF Core migrations:

```
1 dotnet ef database update
```

5. Build and run the application:

```
1 dotnet build
2 dotnet run --project PersonalFinanceManager
```

6. Run tests:

```
1 dotnet test
```

11 Evaluation Criteria Mapping

Criterion	How it is met in this design
Code Readability	Descriptive async method names, single-purpose classes, no magic values.
Design Quality	Strict three-layer architecture; Factory and Repository patterns correctly applied.
Separation of Concerns	Controllers never touch <code>AppDbContext</code> ; services never write to the console; repositories contain no business logic.
Database Integration	EF Core with Fluent API configuration; parameterised LINQ queries; fully async; no raw SQL.
Maintainability	Adding a new entity requires only a new model, configuration class, interface, and repository – no changes to existing code.
Error Handling	Structured <code>ValidationResult</code> objects; over-budget warning propagation; DB failures caught at service layer; clean DI scope disposal on exit.

Appendix: Diagram Index

Figure	Title	Section
1	UML Class Diagram – Full System	3.3
2	Relational Database Schema Diagram	4.2
3	Entity-Relationship Diagram (Chen Notation)	4.4
4	Full Application Flow Diagram	5.1
5	Sequence Diagram – Add Expense with Budget Update	6.1